Blind 75 Recall: Build Brief for Codex

Client lead: Tobi | Implementing engineer: Codex | v1.0 | 24 Oct 2026

1. Summary

Build an Anki-style spaced-repetition app that keeps the NeetCode Blind 75 patterns fresh. The user ticks the Blind 75 problems they have completed; each ticked problem becomes a card; a scheduler tells them every day which problems to re-attempt. The app is iPhone-first, installable, fast, and uses a dark grey, LeetCode-flavoured UI that holds together in landscape.

Success looks like: open the app on an iPhone, see how many problems are due in under a second, finish a session one-handed, and trust the schedule enough to stop thinking about it.

2. How to read this brief

- **MUST** = required for MVP. **SHOULD** = build unless it endangers a MUST. **COULD** = only with spare time.
- **[Default]** marks an assumption the client may override. Build with it, but keep it in one config file or constant so it is cheap to change.
- If Appendix A disagrees with the live NeetCode Blind 75 list (neetcode.io/practice, Blind 75 filter), the live list wins. Record every difference in `docs/seed-notes.md`.

3. Core loop

1. First run: the Library lists the Blind 75 grouped by NeetCode category. The user ticks what they have already completed.
2. Each tick creates a card. The deck is ticked problems only; unticked problems never appear in reviews.
3. Today shows the queue: learning cards due, reviews due, and new cards. The first N, where N is the user's daily goal, are today's picks; the rest wait behind them in the queue.

4. Review: see the problem (title, LeetCode number, difficulty) -> attempt it (in head, on paper, or on LeetCode) -> Show answer (links to NeetCode and LeetCode) -> rate Again / Hard / Good / Easy -> scheduler sets the next due date.
5. When the user solves another Blind 75 problem later, they tick it in the Library and it joins the scheduler immediately.

4. Scope

In (MVP)

- Blind 75 library with completion ticks, grouped by NeetCode category
- Deck = ticked problems; ticking adds the card to scheduling straight away
- Anki-style scheduler: four ratings, learning and relearning steps, ease factor, lapses
- Today queue, review session, answer reveal, undo
- Basic stats: due today, 7-day forecast, streak, retention, cards by maturity
- Settings: daily goal, day rollover hour, first-review timing, steps
- Offline use, installable to the iPhone Home Screen, JSON export and import

Out (v1)

- Accounts, cloud sync, push notifications (need a backend), in-app code editor or judge, AI hints, native iOS build, social features
- NeetCode 150 or other lists (but do not hard-code Blind 75 into the schema; see section 6)

5. Platform and stack [Default]

- Progressive Web App. Installed via Safari > Share > Add to Home Screen, opens standalone with no browser chrome.
- React 18 + TypeScript (strict) + Vite. Styling with Tailwind CSS or plain CSS variables. No heavy component library.
- Storage: IndexedDB (via Dexie) for cards, review log and settings. No backend; all state lives on the device.
- vite-plugin-pwa (Workbox) for manifest and service worker; precache the app shell so it loads fully offline.
- Routing: a lightweight router (React Router or wouter). State: React context plus a small store (Zustand is fine).

- Tests: Vitest for the scheduler and queue builder; Playwright for viewport and flow checks.
- Hosting: any static host (Vercel, Netlify, GitHub Pages). Put deploy steps in the README.
- Budget: initial JS under 150 KB gzipped. Add a dependency only with a clear reason.
- iOS can evict web storage under pressure, so JSON export and import is a MUST, and a gentle backup reminder every 30 days is a SHOULD.

6. Data model

Seed data (static, bundled, versioned JSON; insight, timeComplexity, spaceComplexity and the Card notes field are optional and unused in the MVP, because the card back is links only):

```
Problem {
  id: string              // stable, e.g. 'lc-1'
  lcNumber: number
  title: string
  slug: string            // LeetCode slug
  difficulty: 'easy' | 'medium' | 'hard'
  category: string        // NeetCode category; hidden on the card front
  lists: string[]         // ['blind75'] now; room for 'neetcode150' later
  order: number           // position in NeetCode's list
  premium: boolean        // LeetCode premium flag
  insight?: string        // one sentence: the key idea
  timeComplexity?: string
  spaceComplexity?: string
  leetcodeUrl: string
  neetcodeUrl?: string    // fill from neetcode.io; fall back to leetcodeUrl
}
```

User data (IndexedDB):

```
Card {
  problemId: string       // primary key
  state: 'new' | 'learning' | 'review' | 'relearning'
  ease: number            // starts by difficulty, min 1.3
  intervalDays: number
  dueDay: number          // day index (new and review cards)
  dueAt?: number          // epoch ms (learning and relearning only)
  stepIndex: number
```

```
  reps: number
  lapses: number
  lastReviewedAt?: number
  notes?: string          // the user's own insight
  addedAt: number         // when ticked
  archived: boolean       // unticked; history kept
}

ReviewLog { id, problemId, ratedAt, rating (1-4), stateBefore,
intervalBefore,
            intervalAfter, easeBefore, easeAfter, durationMs }   // append-
only

Settings { dailyGoal, rolloverHour, learningSteps, relearningSteps,
           graduatingIntervalDays, easyIntervalDays, maxIntervalDays,
           firstReview: 'tomorrow' | 'today' }
```

Times are stored as UTC epoch ms. A day index is the number of whole local days since epoch after subtracting the rollover hour (default 04:00, as in Anki).

7. Scheduler (the heart of the app)

7.1 What the four buttons mean for a coding problem

Button	Meaning
Again	Could not solve it, or had to read the solution
Hard	Solved, but needed big hints, struggled for a long time, or recalled the pattern slowly
Good	Solved cleanly, recognised the pattern, reached the optimal approach
Easy	Instant recall of the pattern and optimal solution

Show this guide as a small help sheet from the review screen.

7.2 Defaults [Default] (all configurable, in one config object)

Setting	Value
Learning steps	10 min
Relearning steps	10 min
Graduating interval (Good out of learning)	2 days
Easy interval (Easy out of learning)	5 days
Starting ease	Easy 2.6, Medium 2.5, Hard 2.3
Minimum ease	1.3
Hard multiplier / Easy bonus / interval modifier	1.2 / 1.3 / 1.0
Lapse new-interval multiplier	0.25 (never below 1 day)
Maximum interval	180 days
Leech threshold	4 lapses
Fuzz	plus or minus 5% (min 1 day) on intervals of 3 days or more, seeded by problemId + reps so previews are stable
Daily goal	Daily goal: 3 problems by default, user-set from 1 to 20; a soft target, not a limit (see 7.6 and 7.9)
Day rollover	04:00 local

7.3 Transitions

```
NEW / LEARNING (steps = learningSteps)
  Again: stepIndex = 0;            dueAt = now + steps[0]
  Hard:  stay on stepIndex;        dueAt = now + steps[stepIndex]
  Good:  if last step -> state = review, interval = graduatingInterval,
                        dueDay = today + interval
         else stepIndex++;         dueAt = now + steps[stepIndex]
  Easy:  graduate now, interval = easyInterval

REVIEW   (late = max(0, today - dueDay))
```

```
  Again: lapses++; ease = max(1.3, ease - 0.20)
         interval = max(1, round(interval * 0.25))
         state = relearning; dueAt = now + relearningSteps[0]
  Hard:  interval = max(interval + 1, round((interval + late/4) * 1.2 *
modifier))
         ease = max(1.3, ease - 0.15)
  Good:  interval = max(interval + 1, round((interval + late/2) * ease *
modifier))
  Easy:  interval = max(interval + 1, round((interval + late) * ease * 1.3 *
modifier))
         ease = ease + 0.15          // intervals use the ease from before
this change

RELEARNING
  Again: restart the relearning step
  Good / Easy on the last step: state = review, dueDay = today + interval

AFTER ANY REVIEW-STATE CALC
  apply fuzz -> clamp to maxInterval -> guarantee Hard < Good < Easy by at
least 1 day
```

Missing days never resets progress. Overdue cards simply earn a late bonus through the formulas above.

7.4 Difficulty awareness

Difficulty affects three things only: (1) starting ease, (2) queue tie-breaks (Hard before Medium before Easy at equal overdueness), (3) the badge and stats split. Keep it that simple.

7.5 Interval previews

Each rating button shows its next interval (for example Again <10m, Hard 3d, Good 6d, Easy 11d). Implement `schedule(card, rating, now, settings) -> CardUpdate` as a pure function and compute the previews by calling it four times. The tap applies the same result, so preview and outcome can never disagree.

7.6 Queue builder

1. Learning and relearning cards whose dueAt has passed (allow a 10-minute learn-ahead when nothing else is left).

2. Review cards with dueDay <= today, most overdue first, ties broken by difficulty (Hard first), then seeded random.
3. New cards with dueDay <= today, oldest addedAt first.

The queue is one ordered list. The first dailyGoal items are today's picks; everything after them stays visible as 'Up next'. The goal is a soft target: once the picks are done, 'Keep going' serves the next item with no limit. Anything not done rolls over and keeps its overdue priority. If only waiting learning cards remain, show 'Next card in 7:42' with a 'Show now' override.

7.7 Undo

Undo the last rating: restore the card from the log entry and delete that entry. One level is enough (SHOULD).

7.8 Scheduler tests (MUST)

- Table-driven unit tests for every transition above, including late and on-time reviews.
- Previews equal the actual outcome for all four buttons.
- Fuzz is deterministic for the same card and rep count.
- Day rollover boundary (03:59 vs 04:01) and a daylight-saving change.
- A simulated 365-day run with random ratings never produces NaN, negative intervals, ease below 1.3, or an interval above the maximum.

7.9 Early and extra reviews

The daily goal is soft, so the user can review any deck card at any time, due or not, and as many as they like. A card reviewed before its dueDay is an early review:

- Same four ratings and same ease changes as section 7.3.
- Base interval = days since the last review (minimum 1) instead of intervalDays, with no late bonus. Call the result calc.
- Hard, Good and Easy: dueDay = max(old dueDay, today + calc) and intervalDays = max(old intervalDays, calc). Early practice can never shorten a schedule or inflate it by repetition.
- Again behaves exactly as in 7.3 (lapse, relearning).
- Log it with early: true. It counts towards the daily goal and the streak.
- Tests: Good on an early card leaves dueDay unchanged; Again on an early card lapses it; ten early Goods in a row never move dueDay earlier.

8. Screens

Navigation: bottom tab bar with Today, Library, Stats. Settings is a gear icon on Today.

8.1 Today (home)

- Goal ring showing done / goal (for example 0 / 1), then the total due with a breakdown of Learning / Review / New.
- Primary button 'Start review' (serves today's picks; 'Keep going' then serves more with no limit), pinned in the thumb zone.
- Empty state when nothing is due: 'All caught up. Next up tomorrow: 6 problems.'
- Streak chip. If the deck is empty, a clear call to action: 'Tick the problems you have completed' linking to the Library.
- **Up next queue:** below the button, today's picks first, then everything else in due order (title, difficulty badge, due label). It updates live after every rating. Tapping any row starts that problem out of order. There is no separate calendar screen; the 7-day forecast in Stats covers the future.

8.2 Review session

- Top bar: close, progress against the daily goal (1 / 3, and it keeps counting past the goal), optional stopwatch.
- Front: LeetCode number and title (large), difficulty badge, and an 'Open on LeetCode' button. The category is hidden because recalling the pattern is the exercise.
- Big 'Show answer' button.
- Back: NeetCode and LeetCode link buttons (they open externally) so the user can check their answer, then the four rating buttons with interval previews. No insight text, complexity or notes field in the MVP.
- Keyboard shortcuts (Space, 1-4) for iPad and desktop (SHOULD). Swipe gestures (COULD).

8.2a Rating button colours: Again red, Hard neutral grey, Good green, Easy blue. Hard is deliberately not amber so it never clashes with the Medium badge.

8.3 Library

- Header: 'Completed 23 / 75' with a progress bar split by difficulty.
- Filters: All | Completed | Remaining, plus difficulty chips. Search (COULD).
- Collapsible sections per NeetCode category. Each row: checkbox with a 44 px target, LeetCode number and title, difficulty badge, status (New, Due today, Due in 5d).

- Ticking is instant with an undo toast. Unticking asks 'Remove from deck? History is kept.'
- 'Mark all in category' (SHOULD) for fast onboarding.

8.4 Stats

- Due today, 7-day forecast bars (CSS or SVG, no chart library), streak, 30-day retention (Good + Easy divided by all reviews), cards by maturity (New, Learning, Young under 21d, Mature 21d+).
- Weakest categories by lapse rate and a leech list (SHOULD).

8.5 Settings

- Daily goal (1 to 20, default 3), rollover hour, first-review timing, steps, export and import JSON, reset (double confirm), and an 'Add to Home Screen' help card.

9. UI specification

Mood: LeetCode dark. Dense, calm, quick. Flat surfaces, 1 px borders, 8 px radius, no heavy shadows or gradients. Dark theme only for MVP.

Token	Hex	Use
bg	#1a1a1a	page background (grey base)
surface	#282828	cards, rows
elevated	#333333	sheets, inputs
border	#3e3e3e	dividers
text	#eff1f6	primary text
text-muted	#9ca3af	secondary text
accent	#ffa116	primary button, active tab (LeetCode orange)
easy	#00b8a3	Easy difficulty badge
medium	#ffc01e	Medium difficulty badge
hard	#ff375f	Hard difficulty badge

Token	Hex	Use
again / good / easy-btn	#ef4743 / #2cbb5d / #3b82f6	rating buttons

- **Type:** system font stack (-apple-system, SF Pro), zero font downloads. 16 px base, 20 to 22 px for problem titles, rem units so Dynamic Type works.
- **Spacing and touch:** 4 px grid, every tap target at least 44 x 44 px, primary actions in the bottom 40% of the screen.
- **Motion:** 150 ms or less, respect prefers-reduced-motion, no load animations.
- **Icons:** inline SVG (a lucide subset), no icon font.

Responsive and orientation rules (MUST)

- Viewport: `width=device-width, initial-scale=1, viewport-fit=cover`. Pad with `env(safe-area-inset-*)` on all four sides so the notch, Dynamic Island and home indicator never cover content in either orientation.
- Do not lock orientation (manifest `orientation: any`). The layout must adapt, never break.
- Use `100dvh` with a `100vh` fallback. No fixed pixel heights for screens. Scroll inside a content region, not the body.
- **Portrait phone (under 600 px wide):** single column, bottom tab bar, sticky bottom action area.
- **Landscape phone (height under 500 px):** the tab bar becomes a slim left rail. Today puts the count and Start button side by side. The review screen goes two-column: problem on the left, answer and the four rating buttons (still four across) on the right. Nothing clips, overlaps or hides under the notch.
- **Tablet and desktop (768 px and up):** centre the content, max width 720 px.
- No horizontal scroll at any width from 320 to 1024 px.
- Stop iOS auto-zoom: inputs at 16 px or larger, `-webkit-text-size-adjust: 100%`. Add `touch-action: manipulation` and `overscroll-behavior: none` on the app shell.
- Rotating mid-review preserves state: revealed answer, notes text, timer, queue position.

10. Performance and quality

- Cold load to interactive under 1.5 s on a mid-range iPhone over 4G; effectively instant on repeat visits and offline.
- Lighthouse: Performance 90+, Accessibility 90+, installable PWA.
- Rating tap to next card in under 100 ms: update the UI optimistically and write to IndexedDB asynchronously.
- WCAG AA contrast on the dark theme, VoiceOver labels on every control, visible focus states.
- TypeScript strict, zero console errors, no unused dependencies.

11. Edge cases

- **Empty deck:** guided empty state pointing to the Library.
- **30 problems ticked at once:** new cards queue behind the daily goal rather than flooding one day.
- **Untick then re-tick:** restores the archived card and its history; do not restart it.
- **Clock or timezone change:** schedule by day index; learning cards use absolute timestamps; never crash.
- **App killed mid-session:** rebuild the queue from stored state; no double counting.
- **Two tabs open:** use IndexedDB transactions; last write wins; refresh on storage events (SHOULD).
- **Import:** validate schema version, show preview counts, require confirmation before overwriting.
- **LeetCode Premium problems:** stay in the deck with a small Premium tag; link to NeetCode where available.

12. Acceptance criteria

1. Fresh install: empty deck; Library lists 75 problems with correct title, LeetCode number, difficulty and category.
2. Ticking 3 problems creates 3 cards that show as New starting on the configured day.
3. Rating a review card Good moves its due date exactly per the formulas, and the button preview matched it beforehand.
4. Again sends a card to relearning in 10 minutes and it reappears in the same session.

5. Unticking archives the card; re-ticking restores it with history.
6. Works offline after first load; installs to the Home Screen; reopens in standalone mode.
7. Every screen passes this viewport matrix with no clipping, overlap or horizontal scroll: 375x667, 390x844, 430x932, 667x375, 844x390, 932x430, 820x1180.
8. Export then import into a clean browser reproduces the deck and schedule exactly.
9. Scheduler tests in 7.8 pass, including the 365-day simulation.
10. Lighthouse and bundle-size targets met.
11. With the goal set to 1, Today highlights exactly 1 problem. Rating it shows 'Goal hit', and 'Keep going' still serves the next card. Tapping any queue row lets the user rate that problem too, due or not, with the schedule behaving as in 7.9.

13. Delivery plan

1. **M1:** scaffold, PWA shell, theme tokens, seed data loaded and validated (75 problems).
2. **M2:** scheduler as a pure, fully tested module with no UI, plus the queue builder.
3. **M3:** Library, ticking, persistence.
4. **M4:** Today, Review session, undo.
5. **M5:** Stats, Settings, export and import.
6. **M6:** responsive and landscape pass, accessibility, performance, deploy, README.

Commit once per milestone with a short summary of what changed. Do not start the next milestone until the current one's tests pass. The README must cover run, test, deploy and Add to Home Screen steps.

14. Decisions (1, 2 and 4 confirmed by the client; 3 and 5 are defaults)

#	Decision	Default
1	PWA or native Swift app	PWA
2	Sync	None; local only with JSON export and import
3	When a newly ticked problem first appears	Tomorrow

#	Decision	Default
4	Card back content	Links only: NeetCode and LeetCode buttons, no insight text or notes (client confirmed)
5	Problem lists	Blind 75 only; schema ready for NeetCode 150

Also confirmed by the client: the daily goal is a soft, user-set target (default 3), and any deck card can be reviewed early (sections 7.6 and 7.9).

Appendix A: Blind 75 seed (starting point, verify against NeetCode)

Format: Title (LeetCode number, difficulty E/M/H, P = LeetCode Premium). Categories follow NeetCode's groupings. Total 75. Combination Sum IV (377) is assumed; some Blind 75 copies list Combination Sum (39) instead. Check neetcode.io and log the answer in `docs/seed-notes.md`.

- **Arrays & Hashing (8):** Contains Duplicate (217, E); Valid Anagram (242, E); Two Sum (1, E); Group Anagrams (49, M); Top K Frequent Elements (347, M); Product of Array Except Self (238, M); Longest Consecutive Sequence (128, M); Encode and Decode Strings (271, M, P)
- **Two Pointers (3):** Valid Palindrome (125, E); 3Sum (15, M); Container With Most Water (11, M)
- **Sliding Window (4):** Best Time to Buy and Sell Stock (121, E); Longest Substring Without Repeating Characters (3, M); Longest Repeating Character Replacement (424, M); Minimum Window Substring (76, H)
- **Stack (1):** Valid Parentheses (20, E)
- **Binary Search (2):** Find Minimum in Rotated Sorted Array (153, M); Search in Rotated Sorted Array (33, M)
- **Linked List (6):** Reverse Linked List (206, E); Merge Two Sorted Lists (21, E); Reorder List (143, M); Remove Nth Node From End of List (19, M); Linked List Cycle (141, E); Merge k Sorted Lists (23, H)
- **Trees (11):** Invert Binary Tree (226, E); Maximum Depth of Binary Tree (104, E); Same Tree (100, E); Subtree of Another Tree (572, E); Lowest Common Ancestor of a BST (235, M); Binary Tree Level Order Traversal (102, M); Validate Binary Search Tree (98, M); Kth

Smallest Element in a BST (230, M); Construct Binary Tree from Preorder and Inorder Traversal (105, M); Binary Tree Maximum Path Sum (124, H); Serialize and Deserialize Binary Tree (297, H)

- **Tries (3):** Implement Trie (208, M); Design Add and Search Words Data Structure (211, M); Word Search II (212, H)
- **Heap / Priority Queue (1):** Find Median from Data Stream (295, H)
- **Backtracking (1):** Word Search (79, M)
- **Graphs (6):** Number of Islands (200, M); Clone Graph (133, M); Pacific Atlantic Water Flow (417, M); Course Schedule (207, M); Graph Valid Tree (261, M, P); Number of Connected Components in an Undirected Graph (323, M, P)
- **Advanced Graphs (1):** Alien Dictionary (269, H, P)
- **1-D DP (11):** Climbing Stairs (70, E); House Robber (198, M); House Robber II (213, M); Longest Palindromic Substring (5, M); Palindromic Substrings (647, M); Decode Ways (91, M); Coin Change (322, M); Maximum Product Subarray (152, M); Word Break (139, M); Longest Increasing Subsequence (300, M); Combination Sum IV (377, M)
- **2-D DP (2):** Unique Paths (62, M); Longest Common Subsequence (1143, M)
- **Greedy (2):** Maximum Subarray (53, M); Jump Game (55, M)
- **Intervals (5):** Insert Interval (57, M); Merge Intervals (56, M); Non-overlapping Intervals (435, M); Meeting Rooms (252, E, P); Meeting Rooms II (253, M, P)
- **Math & Geometry (3):** Rotate Image (48, M); Spiral Matrix (54, M); Set Matrix Zeroes (73, M)
- **Bit Manipulation (5):** Number of 1 Bits (191, E); Counting Bits (338, E); Reverse Bits (190, E); Missing Number (268, E); Sum of Two Integers (371, M)